

Experiment - 04.

- Aim: Recurrent Neural Network (RNN) Use the Google stock prices dataset and design a time series analysis and predictions system using RNN.

- THEORY:

- (1) Introduction.

- Time series analysis involves studying historical data collected over time to identify patterns, trends, seasonality and dependencies in order to make future predictions.
- Stock price forecasting is one of the most challenging and important applications of time series modeling due to the sequential and dynamic nature of financial data.
- Traditional machine learning algorithms assume independent data points and fail to capture temporal dependencies. Recurrent Neural Networks (RNNs), however, are specifically designed to handle sequential data by maintaining internal memory of previous inputs. This makes them highly suitable for stock market prediction tasks.

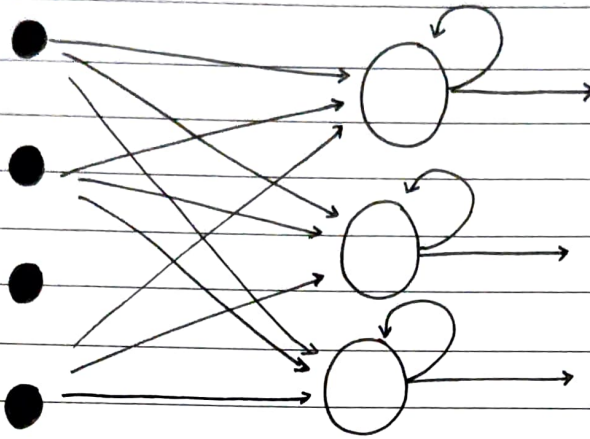
(2). Dataset Description -

- The google stock price dataset contains historical stock market data including:
 - Date.
 - Open price.
 - High price.
 - Low price.
 - Close price.
 - Volume.
- For time series prediction, the closing price is typically used as the target variable.
- Dataset Characteristics -
 1. Sequential time-based data.
 2. Continuous numerical values.
 3. Daily stock price records.
 4. No class labels. (Regression problem).

(3). Why use RNN for time series prediction?

- Stock prices are sequential in nature - today's price depends on previous day's prices. RNNs are capable of capturing temporal dependencies through recurrent connections.
- Advantages of RNN -
 - Maintains memory of previous inputs.
 - Suitable for sequential and time dependent data.
 - Captures temporal patterns and trends.
 - Learns non linear relationships in stock prices.

* RNN Diagram -



(4). Recurrent Neural Network architecture -

- An RNN processes sequences by looping information from one time step to the next.
- Working principle -
At each time step -
 - Input = Current stock price.
 - Hidden state = Information from previous time step.
 - Output = Predicted price.

◦ Mathematically -

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

Where,

- h_t = hidden state at time t .
- x_t = input at time t .
- W_h, W_x = weight matrices.
- b = bias.

(5). Data Preprocessing -

Before training the RNN, the data must be prepared properly. Steps -

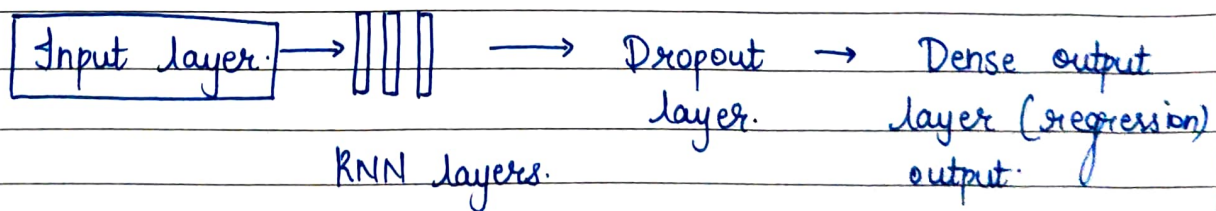
1. Sorting by date.
2. Selecting closing price.
3. Handling missing values.
4. Feature scaling.
 - Min - Max Normalization.
 - Values scaled between 0 and 1.
5. Creating time windows.
 - Example : Use last 60 days to predict next day.

Why windowing?

→ RNN requires sequences as input, Instead of feeding single values, we provide sequences of previous stock prices.

(6). Model Design -

Network architecture -



Activation functions -

- Tanh (commonly used in RNN).
- Linear activation for output layer.

(7). Training configuration -

- Loss function - Mean squared error (MSE)

$$MSE = \frac{1}{n} \sum (y_{true} - y_{pred})^2$$

- Optimiser - adam optimiser.
- Evaluation metrics -
 - Root mean squared error (RMSE)
 - Mean absolute error (MAE).

(8). Model Training.

During Training -

- The RNN learns temporal dependencies in stock prices.
 - Weights are updated using Back propagation Through Time (BPTT).
 - Loss decreases gradually as the model learns patterns.
- Training is performed for multiple epochs until convergence.

(9). Prediction Phase -

After training -

- Test data sequences are fed into the model.
- The model predicts stock prices.
- Predicted values are inverse-transformed to original scale.
- Actual vs. predicted prices are plotted for analysis.

→

(10). Results and observations -

- RNN captures short term trends effectively.
- Predictions follow general stock movement patterns.
- Some deviations occur due to market volatility.
- Performance improves with more training data.

(11). Challenges in stock prediction -

- Market volatility.
- External economic factors.
- News and global events.
- Overfitting risk.

Stock markets are influenced by many external variables not included in historical data, which limits prediction accuracy.

(12). Improvements using advanced RNN variants -

Basic RNN suffers from vanishing gradient problem. Advanced architectures improve performance:

- LSTM - Handles long term dependencies.
 - Most commonly used for stock prediction.
- GRU - Gated recurrent unit - Computationally efficient alternative to LSTM.

* CONCLUSION: Thus we successfully studied and implemented the RNN on stock price data for prediction.